

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Important Notice

The percentage shown reflects a probabilistic assessment and should be treated as an indicator, not a definitive conclusion. Results may vary depending on writing style, subject matter, and document structure. Always use this report as one of several points of reference rather than a sole determining factor.

Guidelines

This report analyzes long-form writing, such as essays, dissertations, and articles. Highlight colors indicate likelihood of AI detection flagging: cyan for high, light purple for moderate, unhighlighted for low.

Non-qualifying content (e.g. bullet points, annotated bibliographies, tables) is not analyzed and may cause the highlighted text to differ from the overall percentage shown. If the document contains non-selectable text, scanned images, or other formatting issues, the score may be inaccurate.

Details

Word Count: 1,368

Document Size: 207.1 KB

Score Interpretation

	0% - 19%	Low likelihood of AI-generated content
	20% - 50%	Moderate likelihood of AI-generated content
	51% - 100%	High likelihood of AI-generated content

Highlight Legend

	High likelihood of being Flagged
	Moderate likelihood of being flagged
	Low likelihood of being flagged



Inspiring Excellence

CSE321 : Operating System

Farhan Sadik

24101406, Sec-09

Introduction

Virtual memory provides the ways to make each process have its own virtual address space. Page tables are used to convert the virtual address into physical address in the operating system. Checking the page table on every memory operation, however, would take a long time; to speed up the process the processor maintains a Translation Lookaside Buffer (TLB), which stores recently performed address translations. If more memory is required in a program than the TLB can support, the TLB miss rate rises and additional page-table walks have to be performed. This can be alleviated if memory objects are made very large (superpages), so that one TLB entry will cover a large area of memory. But with only huge pages, memory would also be wasted in small, or less-used, areas. Thus in modern operating systems there are a number of page sizes.

Address Translation and Performance

TLB Reach and Large Memory Footprints

TLB reach: The size of the maximum memory that can be mapped at a time by the TLB. We can consider it as: $\text{TLB reach} = \text{page size} \times \text{number of TLB entries}$ (Silberschatz et al., 2018). In a fixed-sized TLB, the number of pages covered is relatively small, since the pages are of fixed size, 4KB. Some large applications like databases, and in memory workloads, can consume gigabytes of memory. If the active memory is larger than the TLB coverage, then very useful translations are removed and reloaded again, resulting in more TLB misses. If there is a TLB miss, this means a page-table walk and thus additional memory accesses, making the effective memory access time longer (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

How Huge/Superpages Improve TLB Coverage

The more memory a page references to, the more memory can be indicated by one TLB entry. For instance, a 2 MB page corresponds to the same memory as 512×4 KB pages but only requires 1 translation entry as opposed to 512. So, huge pages can certainly boost the TLB reach, given the same number of TLB entries. This can, in turn, decrease the overhead of virtual-to-physical address translation (Silberschatz et al., 2018; Talluri et al., 1992) due to both fewer translation-lookahead buffers (TLBs) misses and fewer page table walks.

Why Multiple Page Sizes Are Better Than One Large Default

Many people seek to change the default page size in order to make it larger or smaller. If all the pages are made large then TLB coverage will be better, but memory is wasted. A process might only use a small portion of a very large page, and keep the rest of the page unused.

Experimentally Talluri et al. (1992) observed that supporting two page sizes led to a 10% gain, on average, in the working set size, whereas changing the page size from 4KB to 32KB had about a 60% gain, on average. This demonstrates an advantage of selecting the size of the page based on the workload. It is recommended to use normal pages in small (or rarely used) regions, and huge pages in large (densely used) regions. So, having several page sizes offers some compromise between the performance of the TLB and the density of memory (Navarro et al., 2002).

A detailed look at how page size, PTEs, TLB misses and page walks are related

If the size of the memory is the same, smaller pages result in more page table entries as it creates more pages in an address space. All this consumes more space in the page-table and can lead to more work during a page-table walk. Fixed size TLB also may have more miss when the workload is large because of smaller page sizes. Larger pages lead to a smaller number of pages, which helps make TLB reach more likely and has the potential to lower TLB-miss/reaching-PTW rates. That may lead to better memory access performance, but there could be memory-management and waste costs if all pages are larger (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

Huge/Superpage Management and Trade-offs

Formation Through Promotion/Coalescing

To create a "huge page," a set of conventional pages can be treated as a single "huge page. In order to work, the physical pages to be translated must be contiguous to each other and must be aligned correctly; otherwise, the translation of one large page will not be possible for the whole region (Romer et al, 1995). One way to do this is called online promotion, when the system finds a page that is frequently accessed it can be promoted to become a superpage, moving pages as needed to get a good contiguous area of pages for a superpage. One alternative is the reservation based allocation, which determines a large and contiguous physical region to be reserved earlier than the pages are required to be allocated, which limits the need to do later copying (Navarro et al., 2002).

How fragmentations make huge pages not possible

To use huge pages they must be backed by an adequate-sized contiguous physical address. However, even if there is enough free memory you will not necessarily be able to create a huge page. Allocation and de-allocation of free memory may over time result in many small chunks of free memory. This fragmentation of the free space is an external fragmentation, which means that the OS is unable to locate the large aligned block. The usefulness of allocation and

memory-management techniques that maintain or build appropriate contiguous regions are explained in Navarro et al., 2002 and Kwon et al., 2016, respectively.

When Huge Pages Give Little or Negative Benefit

But, using huge pages can sometimes be a hindrance. For programs with active memory that falls in the "normal pages in TLB" region, there may not be much reduction in TLB misses. If only a small fraction of memory is used in the huge page, then they also waste it. Also, a huge page will take additional effort to copy, reserve memory or to manage fragmentation. If the area is not utilized sufficiently these costs will not be recouped. Thus, workloads with high frequencies of small, short-lived or disseminated allocations might not benefit from using large pages (Romer et al., 1995; Kwon et al., 2016). Therefore, huge pages can only be used sparingly, not for every memory region.

Conclusion

Several different page sizes offer a compromise between efficiency of use of memory and performance. Using large pages improves TLB reach, decreases TLB miss and decreases page-table-walk overhead. But don't turn on large pages across the board, as it may be wasting memory and impose extra management overhead. Also note that to create a huge page, the memory needs to be contiguous, and it may be difficult to obtain contiguous memory in a fragmented environment. The research thus favors the following policy: To use huge/superpages in large regularly accessed regions, and small/butterfly pages in small, irregular regions.

References

- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces* (1.10 ed.). Arpaci-Dusseau Books.
- Kwon, Y., Yu, H., Peter, S., Rossbach, C. J., & Witchel, E. (2016). Ingens: Huge page support for the OS and hypervisor. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)* (pp. 705–721). USENIX Association.
- Navarro, J., Iyer, S., Druschel, P., & Cox, A. (2002). Practical, transparent operating system support for superpages. In *Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02)* (pp. 89–104). USENIX Association.
- Romer, T. H., Ohlrich, W., Karlin, A. R., & Bershad, B. N. (1995). Reducing TLB and memory overhead using online superpage promotion. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA '95)* (pp. 176–187). Association for Computing Machinery.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- Talluri, M., Kong, S. I., Hill, M. D., & Patterson, D. A. (1992). Tradeoffs in supporting two page sizes. In *Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA '92)* (pp. 415–424). Association for Computing Machinery. <https://doi.org/10.1109/ISCA.1992.753337>